

BUSN 5000 Project Guide

Chris Cornwell and Abbi Cormier

May 18, 2026

Table of contents

Preface	1
How to use this guide	1
Acknowledgments	1
1 Setup	3
1.1 Create your BUSN 5000 folder	3
1.2 Install R and RStudio	3
1.2.1 Windows	3
1.2.2 macOS	4
1.3 Install the required R packages	5
1.4 Create the subfolders inside Project/	5
1.5 Sort the Pre-project file pack into the right places	5
1.6 Create the .Rproj file	6
1.7 Run the setup check	7
1.8 Common setup mistakes	7
1.8.1 Mistake 1: project_slidedeck.css in the wrong folder	7
1.8.2 Mistake 2: Working from your Downloads folder	7
1.8.3 Mistake 3: Knitting before running the R script	7
1.9 What happens next	8
2 Pre-project	9
2.1 What you'll produce	9
2.2 Workflow	9
2.2.1 Step 1: Open the project in RStudio	9
2.2.2 Step 2: Run LF.R	9
2.2.3 Step 3: Open pre_project.Rmd	10
2.2.4 Step 4: Fill in the YAML author line	10
2.2.5 Step 5: Complete the code chunk on Slide 2	10
2.2.6 Step 6: Fill in the paragraph on Slide 3	11
2.2.7 Step 7: Knit the Rmd to HTML	11
2.2.8 Step 8: Compare your output to the reference	12
2.2.9 Step 9: Save the HTML as PDF	12
2.2.10 Step 10: Submit to Gradescope	12
2.3 Common pre-project mistakes	12
2.4 What happens next	13
3 Main Project	15
3.1 Outline (to be written)	15
4 Progress Check	17
4.1 Outline (to be written)	17
5 Submission	19
5.1 Outline (to be written)	19

6	Common Errors and FAQ	21
6.1	Outline (to be written)	21
7	R and AI Resources	23
7.1	Resources for learning and using R	23
7.1.1	IDEs for R	23
7.1.2	Learning R	23
7.1.3	Workflow	23
7.2	AI resources	24
7.2.1	Working with AI	24
7.2.2	AI for coding	24
7.2.3	Learning more about AI	24

Preface

Welcome to the **BUSN 5000 Project Guide** — the canonical reference for the Pre-project, Project Progress Check, and Final Project for BUSN 5000 (Intro to Data Science for Business and Economics) at the University of Georgia.

This guide covers:

- How to set up your computer and your project folder
- What the pre-project is and how to complete it
- What the main project is and how to complete it
- What the progress check is and how it's graded
- How to submit your work
- How to recognize and recover from common errors

The guide is the single source of truth. Announcements on eLC remind you of upcoming deadlines and point back here for the rules.

How to use this guide

Read the chapters in order the first time through. After that, treat it as a reference — jump to whatever section you need. Each chapter has its own table of contents in the sidebar.

If you find a section that's wrong, unclear, or out of date, please flag it via the “report an issue” link at the top of any page.

Acknowledgments

The course materials in this guide are by **Chris Cornwell** and **Abbi Cormier**, with substantial contributions from the BUSN 5000 teaching team across many semesters.

1Setup

This chapter walks you through getting your computer ready to do BUSN 5000 project work. By the end of it, you should have:

- R and RStudio installed
- All required R packages installed
- A **Project/** folder on your computer with the correct subfolder structure
- Every file from the Pre-project file pack sorted into the right place
- An **.Rproj** file that opens RStudio into the project
- A green report from the setup-check script

Plan to spend 30–45 minutes the first time through.

! Don't skip this

Almost every “my project won't knit” email we get traces back to something here. If setup is right, the rest of the project is much easier. If setup is wrong, every other step compounds the problem.

1.1 Create your BUSN 5000 folder

Before you do anything project-specific, set up a top-level **BUSN 5000** folder on your computer. Put it somewhere you'll remember — your **Documents** folder is a fine default.

Inside **BUSN 5000**, create two subfolders:

```
BUSN 5000/  
  Homework/  
  Project/
```

- **Homework/** is where you'll save homework assignments and other course materials over the semester.
- **Project/** is the self-contained working directory for the Pre-project, Project Progress Check, and Final Project.

The rest of this chapter is about setting up **Project/**. We won't touch **Homework/** here.

1.2 Install R and RStudio

If you haven't already, install both. Follow the instructions for your operating system. Install R before RStudio.

1.2.1 Windows

Step 1: Install R

1. Go to the CRAN R Project website.
2. Click on “Download R for Windows”.
3. Click on “base” and then select the link labeled “Download R x.x.x for Windows” (where x.x.x is the latest version).
4. Open the downloaded `.exe` file and follow the installation prompts, accepting the defaults.

Step 2: Install RStudio

1. Visit the RStudio download page.
2. Click the link to download the Windows installer.
3. Open the downloaded `.exe` file and complete the installation using default settings.

Step 3: Verify the installation

1. Launch RStudio from the Start menu or desktop shortcut.
2. In the Console pane, type `print("Hello, world!")` and press Enter.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.2.2 macOS

First, identify your Mac’s chip. Newer Macs use Apple Silicon (M1, M2, M3, M4, etc.); older ones use Intel processors. You need to know which you have to pick the right R installer.

1. Click the Apple menu (top-left corner) → **About This Mac**.
2. Look at the **Chip** (Apple Silicon) or **Processor** (Intel) line:
 - If it says “Apple M1”, “M2”, “M3”, “M4”, or any Apple-labeled chip, you have **Apple Silicon**. You want the `arm64` installer.
 - If it says “Intel Core ...”, you have an **Intel Mac**. You want the `x86_64` installer.

Step 1: Install R

1. Visit the CRAN R Project website.
2. Click on “Download R for macOS”.
3. Select the latest version matching your chip — either `R-x.x.x-arm64.pkg` (Apple Silicon) or `R-x.x.x-x86_64.pkg` (Intel).
4. Open the downloaded `.pkg` file and follow the installation prompts.

Step 2: Install RStudio

1. Go to the RStudio download page.
2. Click the link to download the macOS version of RStudio.
3. Open the downloaded `.dmg` file and drag the RStudio icon to your **Applications** folder.

Step 3: Verify the installation

1. Open RStudio from your **Applications** folder.
2. In the Console pane, type `print("Hello, world!")` and press Return.
3. If you see `[1] "Hello, world!"`, your installation is successful.

1.3 Install the required R packages

Install all seven packages using the **Packages** pane in RStudio:

1. Select the **Packages** tab in the bottom-right pane.
2. Click **Install** at the top of that pane.
3. In the “Install Packages” box, enter the package names separated by spaces or commas:
here tidyverse modelsummary ggpmisc car ggrepel kableExtra
4. Make sure **Install dependencies** is checked.
5. Click **Install**.

Installation takes a few minutes the first time — let it finish completely.

Tip

You only install packages **once per machine**. After that, the templates load them with `library()` calls inside the Rmd setup chunks, which run every time you knit. Don't put `install.packages()` calls inside an `.Rmd` — that would re-install every time you knit.

1.4 Create the subfolders inside Project/

Inside your BUSN 5000/Project/ folder, create these four subfolders:

- data/
- out/
- r/
- css/

So your skeleton looks like:

```
BUSN 5000/  
  Homework/  
  Project/  
    data/  
    out/  
    r/  
    css/
```

You'll put files into these subfolders in the next step. The `.Rmd` files themselves will live in the `Project/` root, **not** in any of the subfolders.

1.5 Sort the Pre-project file pack into the right places

When you download the Pre-project file pack from eLC, you'll get a flat bucket of 7 files. Your job is to figure out where each one belongs in your `Project/` directory. This isn't busywork — knowing

which file goes where is itself part of what we want you to learn.

Here's the destination for each file in the pack:

File	Goes in
pre_project.Rmd	Project/ (root)
preproject_instructions.pdf	Project/ (root)
LF.R	Project/r/
project_slidedeck.css	Project/css/
pppub24.csv	Project/data/
hhpub24.csv	Project/data/
check_setup_preproject.R	Project/r/

After sorting, your folder should look like:

```
Project/
  pre_project.Rmd
  preproject_instructions.pdf
  data/
    pppub24.csv
    hhpublish24.csv
  out/
    (empty for now - LF.R will write LF.csv here)
  r/
    LF.R
    check_setup_preproject.R
  css/
    project_slidedeck.css
```

1.6 Create the .Rproj file

The **.Rproj** file is what tells RStudio “this folder is an R project.” It anchors all the relative paths the templates use.

To create one:

1. Open RStudio.
2. **File** → **New Project**.
3. Choose **Existing Directory**.
4. Browse to your **Project/** folder. Click **Create Project**.
5. RStudio re-opens with **Project** loaded as the active project.

You'll see **Project.Rproj** appear in the **Project/** root. From now on, **always open RStudio by double-clicking that .Rproj file**. That guarantees your working directory and paths are correct.

1.7 Run the setup check

1. In RStudio's **Files** pane (bottom-right), open the `r/` subfolder.
2. Click `check_setup_preproject.R` to open it in the editor.
3. In the editor, **Select All** (Ctrl+A on Windows, Cmd+A on Mac).
4. Click the **Run** button in the top-right of the editor pane (or press Ctrl+Enter / Cmd+Enter).

The script's output appears in the Console pane. You'll see one line per file: either (good) or with a specific instruction telling you what to fix.

Note

If you see any , fix the listed problems and re-run the script. Don't move past Setup until every line is a .

1.8 Common setup mistakes

In our experience, these are the three setup problems we see most often. Each one is preventable with a few seconds of attention.

1.8.1 Mistake 1: `project_slidedeck.css` in the wrong folder

The CSS file styles your slide deck. If it's anywhere but `Project/css/project_slidedeck.css`, your knitted slides will lose all of their formatting and the submission will be rejected. The setup-check script will catch this, but it's a simple thing to get right on the first pass.

1.8.2 Mistake 2: Working from your Downloads folder

If you double-click `pre_project.Rmd` directly from your Downloads folder, RStudio's working directory is Downloads, not your `Project/` folder. The `here()` paths in the `.Rmd` will resolve to the wrong place and you'll get cryptic "file not found" errors.

Always move the files into your `Project/` folder before opening them. Your Downloads folder is a transit area, not a workspace.

1.8.3 Mistake 3: Knitting before running the R script

The `.Rmd` reads output that the `LF.R` script (and later `cpsmar_e.R`) produces. If you haven't run the script yet, the output file doesn't exist, and the `.Rmd` errors out the moment it tries to read `out/LF.csv`.

Workflow order for each stage:

1. Open the `.Rproj`.
2. Run the relevant R script (`LF.R` for pre-project, `cpsmar_e.R` for main project).
3. *Then* open the `.Rmd` and knit it.

The setup-check script can confirm files exist, but only running the script confirms it actually produced the output the `.Rmd` needs.

1.9 What happens next

You're set up. From here:

- For the **Pre-project**, continue to Chapter 3.
- For the **Main Project** (later in the semester), you'll add more files to the same **Project/** folder — see Chapter 4.
- For the **Project Progress Check**, you continue working on the same main project files — see Chapter 5.
- For supplementary materials on R, RStudio, AI tools, and coding practices, see R and AI Resources.

Your **Project/** directory carries through the entire semester. You won't need to re-do Setup unless you're working from a new machine.

2Pre-project

The Pre-project Exercise is a short, required exercise designed to get you and your computer ready for the Main Project. By completing it correctly, you'll have:

- Installed the required R packages
- Set up the proper directory structure on your computer
- Learned how to knit your `.Rmd` to an HTML slide deck and save it in PDF format
- Earned the **5% of your Project grade** that the Pre-project is worth

! All-or-nothing

The Pre-project is worth **5% of your Project grade**, graded all-or-nothing. If your content is correct **and** your slide deck is 100% compliant with the formatting in `pre_project_knitted_template.html`, you earn the full 5%. Any deviation — content error, formatting error, wrong filename, anything — and you earn 0. There is no partial credit on the Pre-project.

This chapter assumes you've completed Setup. If you haven't created your `Project/` folder, sorted the file pack, created the `.Rproj`, and run `check_setup_preproject.R`, go do that first.

2.1 What you'll produce

A 3-slide PDF slide deck:

1. **Title slide** — auto-generated from the YAML block; just requires your name.
2. **Read `LF.csv` and print its contents** — one code chunk you complete.
3. **Document the contents of `LF.csv`** — one paragraph with six blanks to fill in.

That's it. Three slides.

2.2 Workflow

The order matters. Follow these steps in sequence.

2.2.1 Step 1: Open the project in RStudio

Double-click `Project.Rproj` in your `BUSN 5000/Project/` folder. RStudio opens with the project loaded. You should see "Project" in the top-right corner of the RStudio window.

2.2.2 Step 2: Run `LF.R`

`LF.R` reads the raw CPS data file (`pppub24.csv`), computes the labor force participation rate and its components, and writes the results to `out/LF.csv`. You need to run it before you can knit the `Rmd`.

1. In RStudio's **Files** pane (bottom-right), open the `r/` subfolder.
2. Click `LF.R` to open it in the editor.
3. **Select All** (Ctrl+A on Windows, Cmd+A on Mac).
4. Click the **Run** button (or press Ctrl+Enter / Cmd+Enter).

The console will print a message like `March 2024 LFPR = 62.7...`. If you see it, the script ran. A new file `LF.csv` now lives in your `out/` subfolder.

! Important

You must run `LF.R` **before** you knit `pre_project.Rmd`. The Rmd reads `out/LF.csv`; if that file doesn't exist, the Rmd errors out.

2.2.3 Step 3: Open `pre_project.Rmd`

In the Files pane, click `pre_project.Rmd` (it lives in the `Project/` root, not in any subfolder). It opens in the editor.

2.2.4 Step 4: Fill in the YAML author line

Near the top of the file, find the YAML block (the section between the `---` markers). Look for this line:

```
author: "firstname lastname"
```

Replace `firstname lastname` with your **first and last name**, e.g.:

```
author: "Jane Doe"
```

Don't change anything else in the YAML. This is the most common place students introduce formatting errors.

2.2.5 Step 5: Complete the code chunk on Slide 2

Slide 2 is titled "Read `LF.csv` and print its contents." The code chunk looks like:

```
lf <- read_csv(here("out", "LF.csv"))
print(__)
```

If you've never seen R code before, here's what's happening line by line:

- **Line 1.** `lf <- read_csv(here("out", "LF.csv"))` does three things at once. `read_csv("...")` is a function that reads a CSV file. `here("out", "LF.csv")` builds the file path `out/LF.csv` relative to your project root — this is why you ran `LF.R` first; it created the `LF.csv` that this line is reading. And `lf <- ...` takes the result and stores it in a new **object** named `lf`. The arrow `<-` is R's assignment operator (you'll see it used the same way most other languages use `=`).

- **Line 2.** `print(...)` prints whatever you put inside the parentheses to the slide output. The blank `__` is where you tell R *what* to print.

Replace `__` with the object you want to print. Since you just read the contents into `lf`, the answer is `lf`:

```
lf <- read_csv(here("out", "LF.csv"))
print(lf)
```

Then change the chunk option `eval = FALSE` to `eval = TRUE` at the top of the chunk. This tells R Markdown to actually run the chunk when you knit (it's set to `FALSE` by default so the template knits before you complete it).

Run the chunk (click the small green play button at the top-right of the chunk) to verify it works. You should see the `lf` data frame printed in the console — six columns named `E`, `U`, `LF`, `NILF`, `Pop`, and `LFPR`, with one row of numbers.

Write down those six numbers — you'll use them in the next step.

2.2.6 Step 6: Fill in the paragraph on Slide 3

Slide 3 is titled “Document the contents of `LF.csv`.” It has a paragraph with six blanks (each blank is shown as `_____`). The paragraph reads:

The `LF.csv` file contains the results of the labor force participation rate (LFPR) calculation using the March 2024 CPS produced by the script, `LF.R`. The LFPR (_____) is the ratio of the labor force (_____) to the population (____). The labor force is the sum of the employed (_____) and unemployed (____). The population is the sum of the labor force and those not in the labor force (____). The LFPR is expressed as a percentage.

Each blank goes inside parentheses that follow a specific term — fill in the value of whatever term is named directly before the parentheses, rounded to two decimal places. For example, the first blank follows “the LFPR”, so it gets the LFPR value; the second follows “the labor force”, so it gets the labor force value; and so on.

For reference, here's the mapping from each blank to the matching column in your `lf` data frame:

Blank	Value from <code>lf</code>
LFPR	LFPR
labor force	LF
population	Pop
employed	E
unemployed	U
not in labor force	NILF

2.2.7 Step 7: Knit the Rmd to HTML

With your changes saved, click the **Knit** button at the top of the editor pane.

The knit produces an HTML file in your `Project/` folder. It opens automatically in the RStudio Viewer or your default browser.

2.2.8 Step 8: Compare your output to the reference

The Submission chapter includes a PDF showing exactly what each of the three pre-project slides should look like when rendered correctly — same fonts, same layout, same code chunk styling. Open it side-by-side with your knitted output and confirm they match.

If your output doesn't match exactly, something is wrong. Common causes are listed in the Common Errors chapter.

2.2.9 Step 9: Save the HTML as PDF

You don't knit-to-PDF directly. Instead, you save the HTML output as a PDF using your browser's Save As / Print to PDF function. See the Submission chapter for the exact procedure and browser-specific tips.

2.2.10 Step 10: Submit to Gradescope

Submission filename: `firstname_lastname_pre.pdf` (e.g., `jane_doe_pre.pdf`).

Upload to the **Preproject 0 Exercise** assignment in Gradescope.

2.3 Common pre-project mistakes

In addition to the Setup-stage mistakes in Common Errors (CSS in the wrong folder, working from Downloads, knitting before running the R script), these pre-project-specific mistakes are the ones we see most:

1. **Leaving `eval = FALSE` after completing the code chunk.** If `eval = FALSE`, the chunk doesn't run when you knit, so the output doesn't appear on the slide. Set it to `TRUE` once your code is correct.
2. **Numbers in the paragraph don't match `LF.csv`.** This usually means a student copied numbers from a peer instead of running `LF.R` themselves. The numbers you fill in must come from your own `out/LF.csv` after running your own copy of `LF.R`.
3. **YAML edits beyond the author line.** Changing anything in the YAML besides the author name will break the knit format. Don't touch the `title`, `subtitle`, `date`, `output`, `css`, or any other field.
4. **CSS not correctly placed.** If `project_slidedeck.css` is anywhere other than `Project/css/`, the knit produces an unstyled output that won't match the template. See Mistake 1 in Common Errors.
5. **Saving the wrong way.** The submission must be a PDF generated by saving the *HTML* output, not by knitting directly to PDF. See Submission for the correct workflow.

When in doubt, see Common Errors — most issues have already been documented.

2.4 What happens next

Once you've submitted the Pre-project, you've done the hardest part: your environment works. The Main Project will reuse the same **Project/** folder and the same workflow patterns. Continue to Chapter 4: Main Project when you're ready.

3Main Project

This chapter is the slide-by-slide guide to the BUSN 5000 Project: **Exploring Pay Differences between Women and Men**.

3.1 Outline (to be written)

- Project overview and learning goals
- Data: March 2024 CPS ASEC
- The 35-slide deck structure
- Slide-by-slide instructions (migrated from the current Instructions HTML)
- Point allocations per slide
- Line limits and write-up rules

Note

This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

4Progress Check

The Project Progress Check is an **optional bonus** opportunity that gives you feedback on your project halfway through and rewards progress.

4.1 Outline (to be written)

- What the progress check is and isn't
- Which slides are graded (Title, Academic Honesty, Distribution of earnings by gender, Wages and hours differences, Career log wage profiles, Evolution of the gender wage gap)
- Scoring: 5 points (clean), 3 points (formatting clean / content error), 0 (formatting error)
- How to submit
- How feedback is provided

Note

This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

5Submission

This chapter covers how to submit your work to Gradescope without losing points to formatting.

5.1 Outline (to be written)

- Filename convention (`sectiontime_lastname_firstname_*.pdf`)
- PDF format (landscape, color, formatting preserved)
- The HTML → PDF workflow (not knit-to-PDF)
- Browser-specific save-as-PDF tips
- What “100% compliant with the knitted template” means in practice
- Deadlines and late-submission policy

Note

This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

6Common Errors and FAQ

If something is broken, look here first.

6.1 Outline (to be written)

- The “did setup work?” diagnostic (`check_setup.R`)
- Common error catalog:
 - CSS in the wrong folder
 - `eval=FALSE` left on a completed chunk
 - YAML edited
 - Wrong working directory
 - Missing packages
 - Knit-to-PDF instead of HTML → PDF
 - Incorrect filename on Gradescope
- Communication policy (no content review by email; one thread per issue; no same-issue cross-emails)
- How to ask for help productively

Note

This chapter is a stub. Content will be filled in as part of Wave 2 of the project rebuild.

7R and AI Resources

A curated list of resources for learning and using R, and for working with AI as part of your data-science practice. Nothing here is required reading — but each item below has earned its place. Skim it now, bookmark it, and return as you go.

7.1 Resources for learning and using R

7.1.1 IDEs for R

RStudio remains the most widely used IDE for R, and, like R, it is open-source. Its “Help” tab is a gateway to a wide range of R and RStudio resources. However, Posit (the company behind RStudio) has released Positron, a next-generation IDE built for data science with both R and Python. Positron combines the familiar feel of RStudio with the extensibility of VS Code, and includes built-in AI assistance via Positron Assistant. If you’re comfortable with RStudio, stick with it; if you’re curious about a more modern interface or plan to work across R and Python, Positron is worth exploring.

7.1.2 Learning R

Norm Matloff, a computer scientist and statistician at UC-Davis, has a nice, fast introduction to (base) R. He also has interesting takes on base R vs the Tidyverse and the R vs Python debate in data science. For a more in-depth treatment, consult the widely used *R for Data Science* by Wickham and Grolemund. They will guide you through the Tidyverse. Tidy Data Tutor helps you visualize your data analysis pipelines. James Scott has a great introduction to R that more clearly maps onto BUSN 5000.

You will find a list of econometrics packages for R at the Comprehensive R Archive Network (CRAN). You might prefer the causal inference task view or machine learning task view.

Learning R Markdown or Quarto should go hand-in-hand with learning R. For R Markdown, get started here and keep a cheatsheet handy. You can get started with Quarto here.

7.1.3 Workflow

At least as important as learning R is understanding basic workflow principles. R-bloggers has a beginner’s guide using RStudio Projects. You can find some of the same principles in chapter 2 of Kieran Healy’s *Data Visualization* book (also linked on the course schedule). Matt Gentzkow and Jesse Shapiro provide a fuller take on workflow principles from the perspective of academic social scientists (also posted on eLC). They even provide a manual for doing as they do.

For a perspective specifically on coding practices in data science education, Pruijm, Gîrjău, and Horton (2023) make the case in “Fostering Better Coding Practices for Data Scientists” (*Harvard Data Science Review*) that good coding habits matter even for beginners. Their argument: it’s far easier to learn good practices alongside R than to unlearn bad habits later. They organize their guidance into a practical top-10 list. Worth reading early in your data-science career.

Suffice it to say that Healy’s book is a great introduction to the wonders of ggplot, which is not R, but indispensable to analysis done in R.

7.2 AI resources

7.2.1 Working with AI

My take is that you should view AI chatbots, like ChatGPT, Claude, and Gemini, as exceedingly productive collaborators with whom you should learn to work, much like you would with a human. Unfortunately, there are no secret prompting strategies or special step-by-step manuals (also like with a human). For some practical guidance along these lines, I recommend that you follow Ethan Mollick’s Substack. If you want to swim in the deeper end of the pool, check out the Substack, Don’t Worry About the Vase.

7.2.2 AI for coding

One place gen AI really helps is in coding. You can get coding help from chat interfaces, but you may prefer a more integrated experience. Several options exist:

- Positron Assistant: If you use Positron, this built-in AI assistant understands your R environment, loaded data, and console history. It integrates GitHub Copilot for code completions and Claude for chat and agent mode.
- GitHub Copilot: Works in RStudio, VS Code, and other IDEs. Here is how to get free access as a student. Here are instructions for RStudio integration.
- Claude Code: A command-line tool that can read and modify files in your project directory, run commands, and understand your codebase context. Simon Couch has written useful posts on using Claude Code for R package development.
- Cursor: An AI-first code editor built on VS Code.

Alternatively, you may find you can get by just vibe coding with a chatbot — describing what you want and iterating on the results.

7.2.3 Learning more about AI

To learn more about AI, including a comparison of different models, check out Generative AI for Beginners, a free course offered by Microsoft Cloud Advocates. OpenAI and Anthropic have their own AI academies when you want to step it up.